

Análise Experimental de Algoritmos de Caminho Mínimo: Dijkstra, Bellman-Ford e Otimização de Fronteira via FindPivots

Ribas, R. L. D. E.¹

¹Centro Universitário Dom Helder, Curso de Ciência da Computação, Belo Horizonte, Brazil
enzorochaleitedinizribas@gmail.com

Abstract

Write your abstract here. The abstract should concisely describe the scientific motivation, methods, main results, and relevance to quantum sensing and related physics topics. (keep it max. 150 words)

1 Introdução e Motivação

A Teoria de Grafos é um ramo da matemática e da ciência da computação que estuda as relações entre objetos diversos. Ela é amplamente utilizada como ferramenta de modelagem de problemas reais em diversas áreas do conhecimento, como ciência da computação, engenharias, ciências biológicas, etc..

Um dos problemas mais clássicos e fundamentais na teoria de grafos é o problema do caminho mínimo ou *Single Source Shortest Path (SSSP)*, que consiste em encontrar uma otimização de um caminho entre dois objetos em um grafo, em que esse caminho pode ser caracterizado por um peso, como distância, tempo, custo, etc.. Existem diversos algoritmos para modelar e encontrar soluções para o problema do caminho mínimo, dentre as principais estão os algoritmos de Dijkstra, Bellman-Ford, A* e mais recentemente, o algoritmo proposto por [2] chamado de *Bounded Multi-Source Shortest Path (BMSSP)* que é uma adaptação do algoritmo clássico de Dijkstra que promete uma melhoria de desempenho temporal para um caso específico de grafos.

Este trabalho tem como objetivo realizar uma análise experimental dos algoritmos de Dijkstra, Bellman-Ford e da Sub-rotina FindPivots, que faz parte do núcleo da contribuição de [2], comparando suas eficiências a partir de um *Benchmark* experimental. Através de uma série de experimentos, buscamos entender as vantagens e limitações de cada algoritmo, bem como identificar cenários em que um pode ser preferível ao outro.

2 Fundamentação Teórica

2.1 Teoria de Grafos

Um grafo, denotado por $\mathcal{G}$, é uma estrutura composta por um conjunto ordenado de pares $(V; E)$, em que V é o conjunto dos vértices (ou nós) e E é o conjunto das arestas entre os vértices.

Durante a modelagem de problemas reais, é conveniente construir $\mathcal{G}$ como um grafo **ponderado**, no qual são atribuídos pesos às arestas, representando uma medida de esforço, custo, ou distância entre os dois vértices conectados a elas, e também como um grafo **direcionado**, em que as arestas vão em uma única direção, *e.g.* As linhas de trem de uma rede de metrô de uma cidade, cada nó representa uma estação, e cada aresta representa um caminho da estação de saída até a estação de entrada, e o seu peso o tempo de viagem.

2.2 Algoritmos de Caminho Mínimo

O algoritmo proposto por Edsger Dijkstra em 1959, [1] permanece invicto até os dias atuais, quando o assunto é algoritmos de SSSP para pesos não negativos. Em paralelo, o Algoritmo proposto por [3], conhecido como Algoritmo de Bellman-Ford é a solução clássica que permite a presença de arestas com pesos negativos, embora seja significativamente mais lento que o Dijkstra.

Contrastando com a abordagem gulosa (*greedy*) do Dijkstra, a implementação do algoritmo de Bellman-Ford

adota o princípio de programação dinâmica. A sua arquitetura prescinde de estruturas de dados complexas para ordenação, como filas de prioridade (e.g. *Min Heap*), iterando sistematicamente sobre todas as arestas do grafo e aplicando o processo de relaxamento até $|V| - 1$ vezes.

Por sua vez, o algoritmo de [2] propõe uma abordagem híbrida, combinando a eficiência do Dijkstra com uma estratégia *Divide and Conquer*, que consiste numa poda de fronteira baseada em pivôs, que visa reduzir o número de fontes múltiplas a serem processadas, mitigando o impacto temporal do recálculo isolado. O cerne dessa otimização reside na sub-rotina *FindPivots*, que implementa o Lema 3.2 do [2], um mecanismo de busca local que limita a expansão da fronteira S a um subconjunto estrito de pivôs P , cuja cardinalidade é teoricamente garantida por $|P| \leq \frac{|W|}{k}$, onde W é o conjunto expandido e k é um parâmetro de profundidade.

3 Experimentos

O presente trabalho, alinhado às diretrizes de [5], propõe a implementação e a análise comparativa de desempenho dos algoritmos clássicos para o problema de Caminhos Mínimos de Fonte Única ou *Single Source Shortest Path (SSSP)*, notadamente Bellman-Ford e Dijkstra. Além de estabelecer este *benchmark* referencial, o estudo apresenta uma implementação e uma análise empírica crítica da sub-rotina *Find Pivots* (Lema 3.2), que constitui o núcleo da recente contribuição teórica de [2].

Vale ressaltar que o escopo deste trabalho é estritamente experimental, focando na implementação e na avaliação de desempenho dos algoritmos mencionados e não representa uma implementação integral do algoritmo BMSSP proposto por [2]. A análise teórica detalhada, incluindo a prova formal do Lema 3.2 e a demonstração da complexidade assintótica do BMSSP, está além do escopo desse relatório e é recomendada para leitura direta na fonte original, e fica como sugestão para trabalhos futuros.

3.1 Ambiente Experimental

Todas as implementações do *benchmark* foram integralmente desenvolvidos na linguagem Python (versão 3.14.3), escolhida por sua simplicidade e alta velocidade de desenvolvimento, e estão disponibilizadas em [4]. Todos os testes foram realizados em uma máquina com as seguintes especificações:

Table 1: Ambiente de Teste e Versões. Fonte: Próprio autor.

Linguagem	Versão
Python	3.14.3
Hardware	
Processador	Intel(R) Core(TM) i3-4010U CPU @ 1.70GHz Cores
Memória RAM	16 GB
GPU	Intel Corporation Haswell-ULT Integrated Graphics Controller (rev 09)
S.O.	Arch Linux 6.19.11-arch1-1

O escopo de validação empírica foi metodologicamente estruturado em dois experimentos principais, detalhados nas subseções a seguir.

3.2 Experimento 1: Comparativo de Desempenho (Dijkstra vs. Bellman-Ford) em Grafos Aleatórios

3.2.1 Metodologia

O primeiro experimento tem como objetivo avaliar o comportamento e o tempo de execução dos algoritmos clássicos sob diferentes densidades topológicas. Para garantir a reprodutibilidade e o controle das instâncias, desenvolveu-se um gerador parametrizável de grafos aleatórios direcionados.

As instâncias de teste foram dimensionadas com o número de vértices variando no conjunto

$$N \in \{100, 500, 1000, 5000, 10000\}$$

. Para avaliar a sensibilidade dos algoritmos à densidade das arestas, os grafos foram segmentados em duas categorias:

- **Grafos Esparsos:** Onde o número de arestas cresce linearmente em função dos vértices, definido por $E = k_e \cdot N$, sendo k_e uma constante de ramificação.
- **Grafos Densos:** Onde o número de arestas aproxima-se de uma fração do limite quadrático do grafo completo, definido por $E = \lfloor \frac{N(N-1)}{k_d} \rfloor$, onde k_d atua como um fator redutor de densidade global.

O cruzamento destas topologias permite evidenciar o impacto estrutural da fila de prioridade do algoritmo de Dijkstra em contraposição ao relaxamento massivo de arestas do Bellman-Ford.

3.2.2 Implementação do Algoritmo de Dijkstra

A implementação do algoritmo de Dijkstra foi desenhada para maximizar a eficiência empírica no modelo de computação clássico. Para assegurar a complexidade estrutural de $O((V + E) \log V)$, adotou-se uma fila de prioridade baseada em um *Binary Heap*, utilizando a biblioteca nativa `heapq` da linguagem Python. O controle de estado dos vértices foi projetado por meio de tabelas de espalhamento (*hash tables* implementadas via `dict`), o que garante tempo de acesso amortizado $O(1)$ para a recuperação de distâncias correntes, verificação de nós visitados e rastreamento de predecessores.

A lógica matemática de relaxamento de arestas e extração da fila de prioridade está formalizada no pseudocódigo do Algoritmo 1, o qual reflete com exatidão a estratégia construída para os experimentos empíricos.

Algorithm 1 Algoritmo de Dijkstra com *Binary Heap*

Require: Grafo $G = (V, E)$, Vértice origem s

Ensure: Vetor de distâncias d , Vetor de predecessores π , Tempo de execução Δt

```

1: Iniciar cronômetro
2:  $d[v] \leftarrow \infty, \forall v \in V$ 
3:  $visited[v] \leftarrow \text{Falso}, \forall v \in V$ 
4:  $\pi[v] \leftarrow \text{Nulo}, \forall v \in V$ 
5:  $d[s] \leftarrow 0$ 
6: Inicializar fila de prioridade mínima  $Q$ 
7:  $Q.push((0, s))$ 
8: while  $Q$  não estiver vazia do
9:    $(d_u, u) \leftarrow Q.pop()$ 
10:  if  $visited[u]$  then
11:    continue
12:  end if
13:   $visited[u] \leftarrow \text{Verdadeiro}$ 
14:  for all vizinho  $v$  de  $u$  com aresta de peso  $w$  do
15:    if  $w \leq 0$  then ▷ Arestas de peso nulo ou negativo são ignoradas
16:      continue
17:    end if
18:     $new\_distance \leftarrow d_u + w$ 
19:    if  $new\_distance < d[v]$  then
20:       $d[v] \leftarrow new\_distance$ 
21:       $\pi[v] \leftarrow u$ 
22:       $Q.push((d[v], v))$ 
23:    end if
24:  end for
25: end while
26: Parar cronômetro e calcular  $\Delta t$ 
27: return  $d, \pi, \Delta t$ 

```

3.2.3 Implementação do Algoritmo de Bellman-Ford

Para adequar o algoritmo de Bellman-Ford para grafos do mundo real e mitigar o seu custo assintótico de pior caso de $O(V \cdot E)$, a implementação foi projetada com um mecanismo de *Early Stopping* (Parada Antecipada). Este mecanismo monitora o estado de mutabilidade da fronteira: se uma iteração completa sobre o conjunto de vértices não produzir nenhuma atualização no vetor de distâncias, infere-se matematicamente que os caminhos mínimos já convergiram para a solução ótima, permitindo a interrupção prematura do laço principal.

O detalhamento da lógica iterativa, incluindo a condição de otimização, encontra-se formalizado no Algoritmo 2.

Algorithm 2 Algoritmo de Bellman-Ford (com *Early Stopping*)

Require: Grafo $G = (V, E)$, Vértice origem s

Ensure: Vetor de distâncias d , Vetor de predecessores π , Tempo de execução Δt

```

1: Iniciar cronômetro
2:  $d[v] \leftarrow \infty, \forall v \in V$ 
3:  $\pi[v] \leftarrow \text{Nulo}, \forall v \in V$ 
4:  $d[s] \leftarrow 0$ 
5: for  $i = 1$  to  $|V| - 1$  do
6:    $updated \leftarrow \text{Falso}$ 
7:   for all  $u \in V$  do
8:     if  $d[u] = \infty$  then
9:       continue
10:    end if
11:    for all vizinho  $v$  de  $u$  com aresta de peso  $w$  do
12:      if  $w \leq 0$  then ▷ Arestas de peso nulo ou negativo são ignoradas
13:        continue
14:      end if
15:      if  $d[u] + w < d[v]$  then
16:         $d[v] \leftarrow d[u] + w$ 
17:         $\pi[v] \leftarrow u$ 
18:         $updated \leftarrow \text{Verdadeiro}$ 
19:      end if
20:    end for
21:  end for
22:  if not  $updated$  then
23:    break ▷ Convergência alcançada prematuramente
24:  end if
25: end for
26: Parar cronômetro e calcular  $\Delta t$ 
27: return  $d, \pi, \Delta t$ 

```

3.2.4 Decisões de Projeto e Desafios Encontrados

Inicialmente foi adotada uma abordagem com matrizes de adjacência para a geração dos grafos por meio do algoritmo parametrizável para geração de grafos aleatórios, mas com o crescimento do número de vértices, a complexidade espacial de $O(V^2)$ tornou-se proibitiva, especialmente para grafos densos, o tempo para leitura e escrita dos dados se tornou inviável (cerca de 5-10 minutos para grafos com 10.000 vértices). Para contornar esse desafio, a implementação foi adaptada para utilizar listas de adjacência, que oferecem uma representação mais compacta e eficiente para grafos esparsos, reduzindo a complexidade espacial para $O(V + E)$.

O algoritmo *get_neighbors* na implementação inicial tinha um tempo de execução $O(V)$ para cada vértice analisado, criando um *overhead* significativo. Já na implementação final com listas de adjacência, o tempo de execução para obter vizinhos de um grafo se torna $O(1)$ ou $O(k)$ para vértices de k vizinhos.

Além disso, para evitar a presença de arestas paralelas e loops, uma estrutura de dados *seen_nodes* foi introduzida durante a geração dos grafos, garantindo a unicidade das arestas e a integridade estrutural do grafo. Apesar de não ter funcionado inicialmente, o problema foi identificado e corrigido, sua causa era que para grafos com o número de arestas maior que o máximo teórico para um grafo simples ($E > \frac{N(N-1)}{2}$), o algoritmo de geração entrava em um loop infinito tentando gerar arestas únicas, a solução foi limitar o número de arestas para o máximo teórico permitido para um grafo simples.

Ademais, durante os *batches* de *benchmarking*, foi necessário introduzir uma lógica de *caching*, para evitar que os grafos fossem gerados todas as vezes que o algoritmo fosse rodado, economizando uma quantidade significativa de tempo.

A implementação de ambos os algoritmos exigiu uma série de decisões de projeto para garantir a eficiência e a correção. Para o algoritmo de Dijkstra, a escolha da estrutura de dados para a fila de prioridade foi crucial, optando-se por um *Binary Heap (heappq)* para equilibrar a complexidade de inserção e extração. Já para o Bellman-Ford, a implementação do mecanismo de *Early Stopping* foi fundamental para evitar iterações

desnecessárias em casos onde os caminhos mínimos já haviam convergido.

3.2.5 Discussão e Resultados

Os resultados empíricos obtidos no Experimento 1 corroboram a fundamentação teórica acerca da complexidade assintótica de ambos os algoritmos. A Tabela 2 apresenta os tempos de execução (*wall-clock time*) em milissegundos para grafos esparsos e densos, variando o número de vértices (N) de 100 a 10.000.

Table 2: Tempos de execução (ms) de Dijkstra e Bellman-Ford em grafos aleatórios.

Topologia	N (Vértices)	E (Arestas)	Dijkstra (ms)	Bellman-Ford (ms)	Razão (BF/Dijk)
Esperso	100	200	0.25	1.28	5.12x
	100	300	0.27	1.31	4.85x
	100	400	0.38	1.67	4.39x
	500	1000	1.29	10.33	8.01x
	500	1500	2.49	13.82	5.55x
	500	2000	1.93	16.35	8.47x
	1000	2000	2.69	27.94	10.39x
	1000	3000	4.04	29.11	7.21x
	1000	4000	5.93	31.54	5.32x
Denso	100	2475	0.88	8.62	9.80x
	100	4950	1.72	14.00	8.14x
	500	62375	19.76	183.47	9.28x
	500	124750	35.80	395.86	11.06x
	1000	249750	106.91	865.83	8.10x
	1000	499500	165.30	1325.46	8.02x

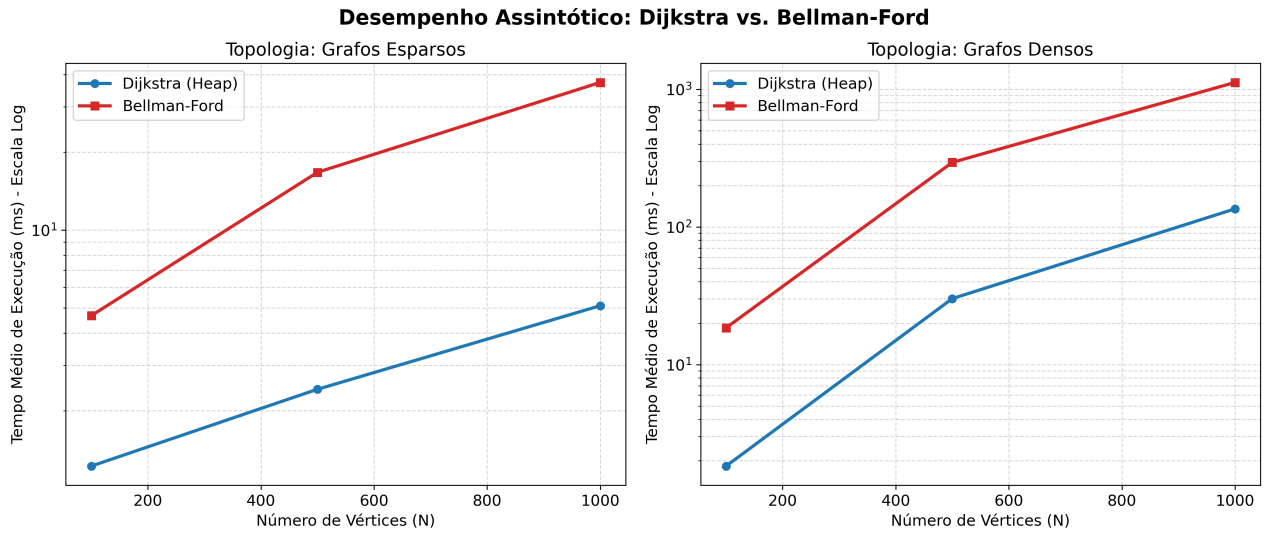


Figure 1: Example figure caption. Figures should be clear and readable.

O resultado do experimento mostra a clara superioridade de desempenho do algoritmo de Dijkstra em relação ao Bellman-Ford em todos os cenários testados, corroborando integralmente a teoria de complexidade assintótica.

Para grafos esparsos, onde o número de arestas cresce de forma aproximadamente linear em relação aos vértices ($E = O(V)$), a implementação do Dijkstra já se mostra de 4 a 10 vezes mais rápida. Contudo, devido à baixa densidade, os tempos absolutos de ambos os algoritmos mantêm-se na casa das dezenas de milissegundos (atingindo no máximo 31.54 ms para $N = 1000$ no Bellman-Ford), o que torna o *overhead* tolerável em aplicações de pequena escala.

A verdadeira divergência arquitetural revela-se no cenário de grafos densos ($E = O(V^2)$), visualizada claramente na Figura 1. Com o aumento quadrático das arestas, o gargalo iterativo do Bellman-Ford torna-se crítico. Na instância mais rigorosa do teste ($N = 1000$ e $E = 499500$), o algoritmo de Dijkstra convergiu a árvore de caminhos mínimos em apenas 165.30 ms, alavancado pela eficiência de extração $O(\log V)$ da sua fila de prioridade (*Binary Heap*). Em contrapartida, o Bellman-Ford consumiu 1325.46 ms (mais de 1.3 segundos).

Essa disparidade é a prova empírica da diferença entre a complexidade $O((V+E) \log V)$ do Dijkstra e o limite de pior caso $O(V \cdot E)$ do Bellman-Ford. Mesmo assumindo que a implementação testada possui a otimização de *Early Stopping* — que interrompe laços desnecessários —, a necessidade inerente do Bellman-Ford de relaxar cegamente meio milhão de arestas múltiplas vezes destrói a sua escalabilidade temporal.

Conclui-se, a partir dos dados extraídos, que para qualquer topologia de rede desprovida de arestas com pesos negativos, o emprego do algoritmo de Bellman-Ford introduz uma penalidade computacional injustificável frente à abordagem gulosa e estruturada do Dijkstra.

3.3 Experimento 2: Análise de Sensibilidade da Poda de Pivôs (Lema 3.2)

O segundo experimento foca estritamente na mecânica de extração de pivôs proposta pela sub-rotina *Find-Pivots*. Conforme o escopo primário estabelecido para este trabalho, a avaliação inicial concentrou-se em instâncias de grafos esparsos. Foi experimentalmente testado como a implementação parcial e adaptada do algoritmo *BMSSP* reage a variação do parâmetro de profundidade k .

3.3.1 Módulo de Avaliação de Fronteira: Partial Dijkstra

O papel da rotina *Partial Dijkstra* no ecossistema do *BMSSP* é atuar como um motor de inicialização do conjunto de fronteira (S). Em vez de computar os caminhos mínimos para todo o grafo, a execução é prematuramente abortada quando um número predeterminado de vértices (*extraction_limit*) é consolidado.

A principal responsabilidade arquitetural desta função é retornar o limite de distância estrito da fronteira (B) e identificar o conjunto de vértices de borda (S). Tais vértices consistem nos nós cujas distâncias foram provisoriamente relaxadas, porém não exploradas, isolando o estado do grafo no exato ponto onde a extração de pivôs (Lema 3.2) deve iniciar as suas operações de busca em largura. O pseudocódigo é apresentado no Algoritmo 3.

Algorithm 3 Dijkstra Parcial (Inicialização de Fronteira)

Require: Grafo $G = (V, E)$, Origem s , Limite L (*extraction_limit*)
Ensure: Vetor de distâncias parciais d , Limite B , Conjunto de fronteira S

- 1: Inicialização das estruturas d , *visited* e Q (similar ao Dijkstra padrão)
- 2: *processed_count* $\leftarrow 0$
- 3: *last_dist* $\leftarrow 0.0$
- 4: **while** Q não estiver vazia **e** *processed_count* $< L$ **do**
- 5: $(d_u, u) \leftarrow Q.\text{pop}()$
- 6: **if** *visited*[u] **then**
- 7: **continue**
- 8: **end if**
- 9: *visited*[u] $\leftarrow$ Verdadeiro
- 10: *processed_count* $\leftarrow$ *processed_count* + 1
- 11: *last_dist* $\leftarrow d_u$
- 12: Realizar relaxamento padrão das arestas vizinhas
- 13: **end while**
- 14: $S \leftarrow \emptyset$
- 15: **for all** $v \in V$ **do**
- 16: **if** não *visited*[v] **e** $d[v] < \infty$ **then**
- 17: $S \leftarrow S \cup \{v\}$
- 18: **end if**
- 19: **end for**
- 20: $B \leftarrow$ *last_dist*
- 21: **return** d, B, S

3.3.2 Mecânica de Poda de Super-Fontes: Lema 3.2 (Find Pivots)

O cerne do estudo de ablação proposto neste trabalho reside no algoritmo de detecção de pivôs (*FindPivots*), baseado no Lema 3.2 de Duan et al. (2025). O objetivo da sub-rotina é limitar a profusão da fronteira S , convertendo-a num subconjunto estrito de pivôs P cuja cardinalidade teórica assegura $|P| \leq \frac{|W|}{k}$, mitigando o impacto exponencial do recálculo isolado de fontes múltiplas.

A arquitetura do algoritmo implementado se segmenta em quatro fases síncronas. Inicialmente, instaura-se uma Busca em Largura de Relaxamento (*BFS*) limitada estritamente a k passos, monitorando a variável de segurança teórica: a interrupção prematura é decretada assim que o total de vértices explorados excede $|W| > k \cdot |S|$. Superada a busca e a proteção algorítmica, o grafo instaura a construção topológica da floresta e identifica via Busca em Profundidade (*DFS*) as raízes, os tamanhos da árvore subordinada, e filtra os pivôs legíveis. O pseudocódigo que mapeia a execução deste pipeline está delineado no Algoritmo 4.

Algorithm 4 Busca e Poda de Pivôs (Lema 3.2)

Require: Fronteira S , Distâncias correntes $\hat{d}$, Profundidade restritiva k , Limite global B

Ensure: Conjunto de Pivôs P , Conjunto Expandido W

```

1:  $W \leftarrow S$ 
2:  $layers \leftarrow [S]$  ▷ Fase 1: Busca Local de  $k$  passos
3: for  $i = 1$  até  $k$  do
4:    $next\_layer \leftarrow \emptyset$ 
5:   for all  $u \in layers[último]$  do
6:     for all vizinho  $v$  de  $u$  com aresta de peso  $w > 0$  do
7:        $new\_dist \leftarrow \hat{d}[u] + w$ 
8:       if  $new\_dist < \hat{d}[v]$  e  $new\_dist \leq B$  then
9:          $\hat{d}[v] \leftarrow new\_dist$ 
10:        if  $v \notin W$  then
11:           $W \leftarrow W \cup \{v\}$ 
12:           $next\_layer \leftarrow next\_layer \cup \{v\}$ 
13:        end if
14:      end if
15:    end for
16:  end for
17:  if  $|W| > k \cdot |S|$  then ▷ Gatilho de Segurança (Pânico)
18:    return  $S, W$ 
19:  end if
20:  if  $next\_layer = \emptyset$  then
21:    break
22:  end if
23:  Adicionar  $next\_layer$  a  $layers$ 
24: end for ▷ Fases 2 a 4: Reconstrução das Florestas, Extração DFS e Filtro de Tamanho  $\geq k$ 
25: (...) ▷ Construção dos dicionários  $pred\_in\_f$ ,  $tree\_size$  a partir de  $W$ 
26:  $P \leftarrow \{r \in \text{raízes} \mid r \in S \text{ e } tree\_size[r] \geq k\}$ 
27: return  $P, W$ 

```

3.3.3 Integração Sistêmica: Dijkstra Otimizado com Pivôs

A sub-rotina final integra o fatiamento topológico do *Partial Dijkstra* aos resultados extraídos pelo *Find Pivots*. A responsabilidade desta camada é calcular as rotas isoladas de Dijkstra engatilhadas de cada super-fonte em P , iterar sobre a matriz de adjacência residual de distâncias unificadas, e combinar a topologia mais performática para garantir que as travessias resultantes contornam gargalos temporais. O Algoritmo 5 mapeia a orquestração do *pipeline* combinatório e os respectivos contadores de validação.

Algorithm 5 Dijkstra Otimizado via Pivôs Unificados

Require: Origem s , Limite L , Profundidade k **Ensure:** Distâncias combinadas, Dicionário de Métricas, $Total_Time$, $Pivot_Time$

```
1: Iniciar cronômetro global
2:  $\hat{d}, B, S \leftarrow \text{PartialDijkstra}(s, L)$ 
3: Salvar estado imutável de  $\hat{d}_{state} \leftarrow \hat{d}.copy()$ 
4: Iniciar cronômetro parcial ( $pivots$ )
5:  $P, W \leftarrow \text{FindPivots}(\infty, S, \hat{d}_{state}, k)$ 
6: Finalizar cronômetro parcial ( $pivots$ )
7: Inicializar lista  $pivot\_distances \leftarrow []$ 
8: for all  $p \in P$  do
9:    $p\_dist, \dots \leftarrow \text{Dijkstra}(p)$  ▷ Executar roteamento isolado
10:  Adicionar  $p\_dist$  em  $pivot\_distances$ 
11: end for
12: Matriz  $combined\_dist \leftarrow \hat{d}.copy()$ 
13: for all pivô  $p \in P$  em  $pivot\_distances$  do
14:    $dist\_to\_pivot \leftarrow \hat{d}[p]$ 
15:   for all vértices  $v \in V$  do
16:      $path\_via\_p \leftarrow dist\_to\_pivot + p\_dist[v]$ 
17:     if  $path\_via\_p < combined\_dist[v]$  then
18:        $combined\_dist[v] \leftarrow path\_via\_p$ 
19:     end if
20:   end for
21: end for
22: Finalizar cronômetro global
23: Gerar dicionário de métricas baseadas no tamanho de  $|S|, |W|$  e  $|P|$ .
24: return  $combined\_dist$ , Métricas,  $Total\_Time$ ,  $Pivot\_Time$ 
```

3.3.4 (Extra): Comportamento em Escala e Exaustão do Grafo ($N \in [100, 10000]$ e $k \leq 128$)

Além do experimento dentro do contexto principal do trabalho $N = 1000$ e $k \leq 16$ foi feita uma análise com a expansão dos testes para cenários mais amplos (variando N de 100 a 10.000 e forçando o limite de k até 128). Essa análise extra expôs alguns aspectos interessantes do limite de segurança imposto pelo Lema 3.2.

3.4 Discussão e Resultados**3.4.1 Cenário Principal:** $N = 1000$, $m = 2000$ com $k \in [2, 16]$

O cenário primário focou em instâncias de $N = 1000$ vértices, com o parâmetro de profundidade k variando de 2 a 16.

Table 3: Análise de Sensibilidade da Poda (Lema 3.2) em Grafos Esparsos $N=1000$. Tempos convertidos para milissegundos (ms).

N (Vértices)	k	$ S $	Limite ($k \cdot S $)	$ W $	$ P $	Dijkstra Base (ms)	Total BMSSP (ms)	Penalidade
1000	2	34	68	99	34	3.16	115.01	36.4x
	4	34	136	215	34	3.16	104.02	32.9x
	6	34	204	215	34	3.16	101.23	32.0x
	8	34	272	371	34	3.16	95.78	30.3x
	10	34	340	371	34	3.16	92.59	29.3x
	12	34	408	531	34	3.16	92.72	29.3x
	14	34	476	531	34	3.16	96.57	30.6x
	16	34	544	643	34	3.16	120.31	38.1x
	32	34	1088	755	9	3.16	45.79	14.5x
	64	34	2176	755	1	3.16	8.62	2.7x
	128	34	4352	755	0	3.16	5.84	1.8x

Os painéis a seguir detalham a dinâmica interna do Lema 3.2 durante o processo de extração de super-fontes. A Figura 2 introduz a relação direta entre o parâmetro restritivo de profundidade e a retenção de pivôs.

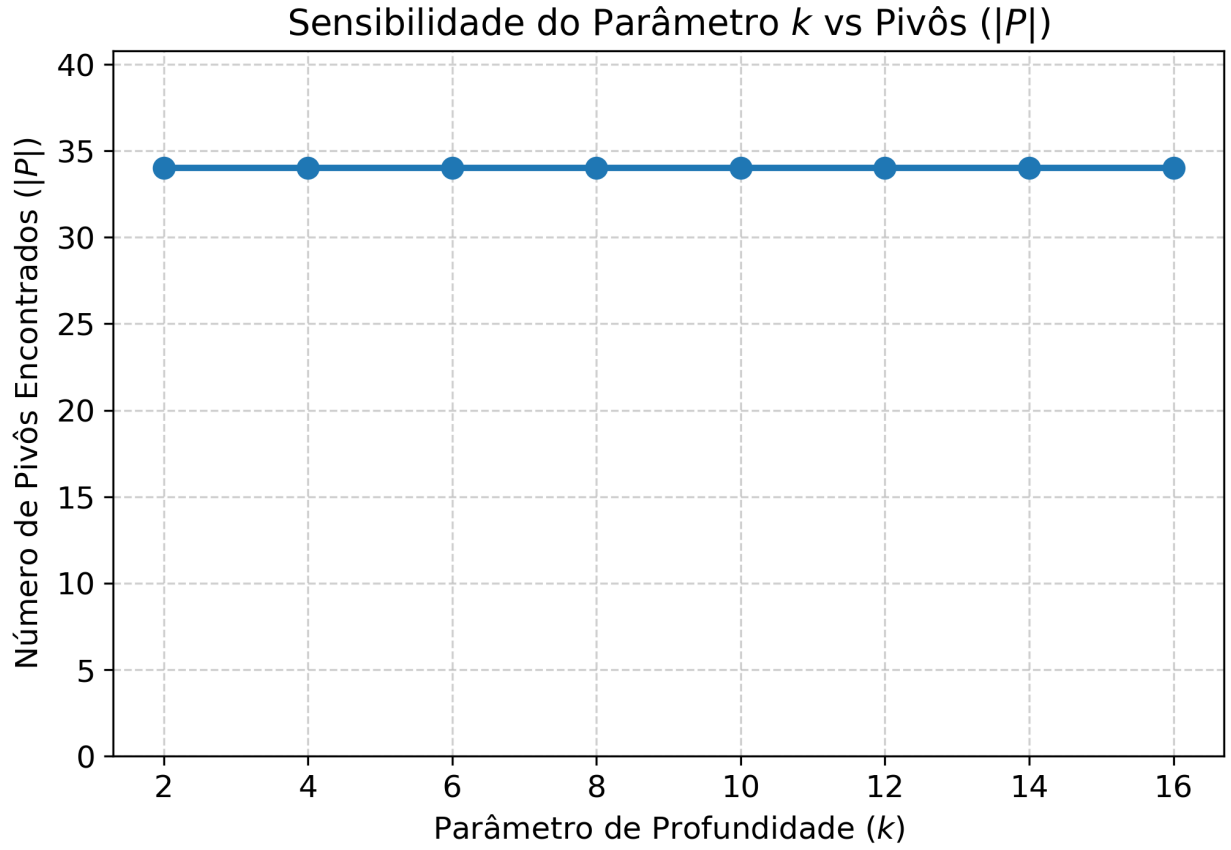


Figure 2: Decrescimento Teórico: Impacto do parâmetro de profundidade k no número de pivôs resultantes $|P|$.

Em teoria, à medida que k aumenta, o filtro local torna-se mais rigoroso e o número de pivôs $|P|$ deveria decair de forma acentuada. No entanto, a estabilidade da curva para valores de $k \leq 16$ atesta que a restrição de segurança ($|W| > k \cdot |S|$) foi acionada precocemente. A BFS depara-se rapidamente com vértices de alto grau (*Hubs*), abortando a poda e forçando o algoritmo a retornar a fronteira intacta.

Essa incapacidade de redução reflete-se diretamente na métrica de economia computacional, explicitada na Figura 3.

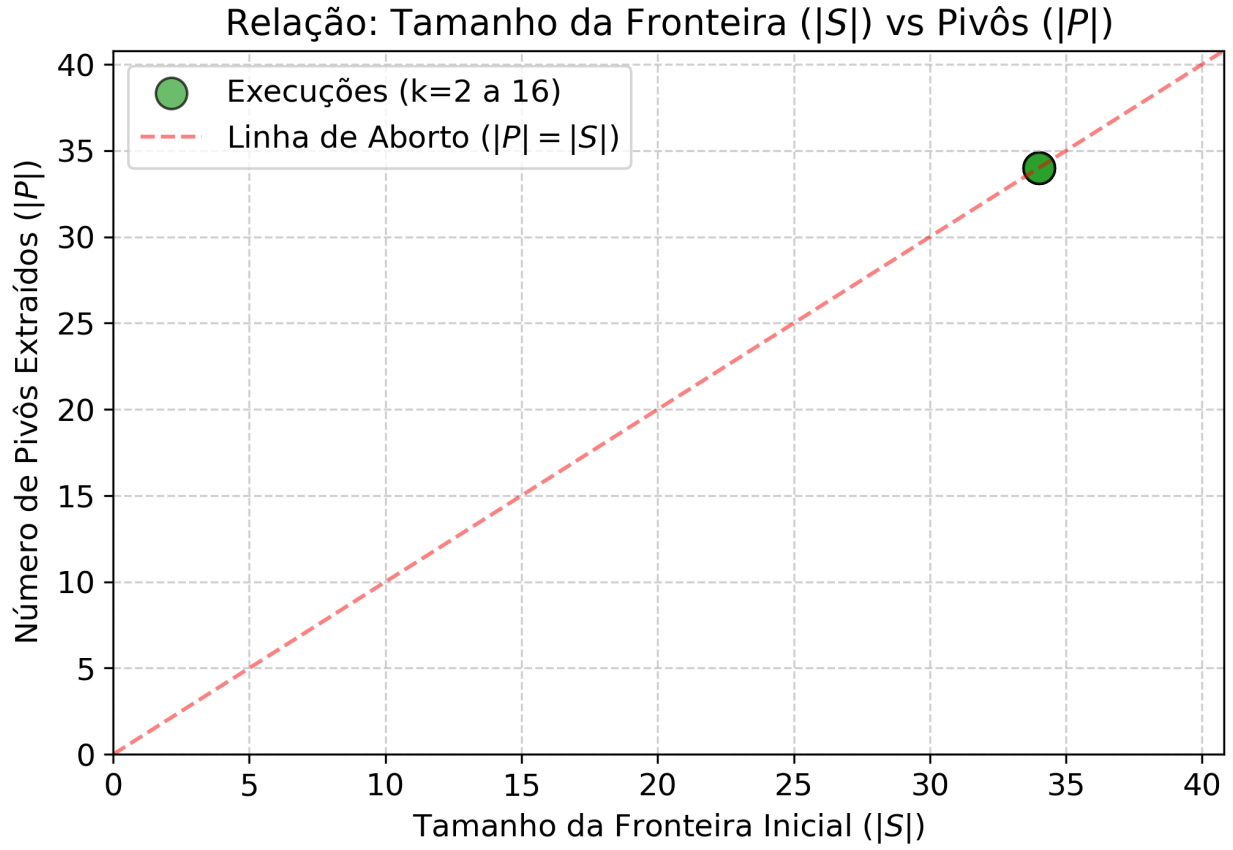


Figure 3: Dinâmica de Poda: Comparativo entre a cardinalidade da fronteira de entrada $|S|$ e a retenção final de pivôs $|P|$.

A área divergente entre as curvas representaria a eliminação de redundâncias. O colapso (sobreposição) das linhas para instâncias operacionais evidencia a ineficácia topológica da estratégia em grafos aleatórios. O algoritmo trabalha para filtrar a fronteira, mas acaba por extrair $|P| \approx |S|$.

O peso deste esforço redundante é quantificado temporalmente na Figura 4, que mapeia o custo de pré-processamento.

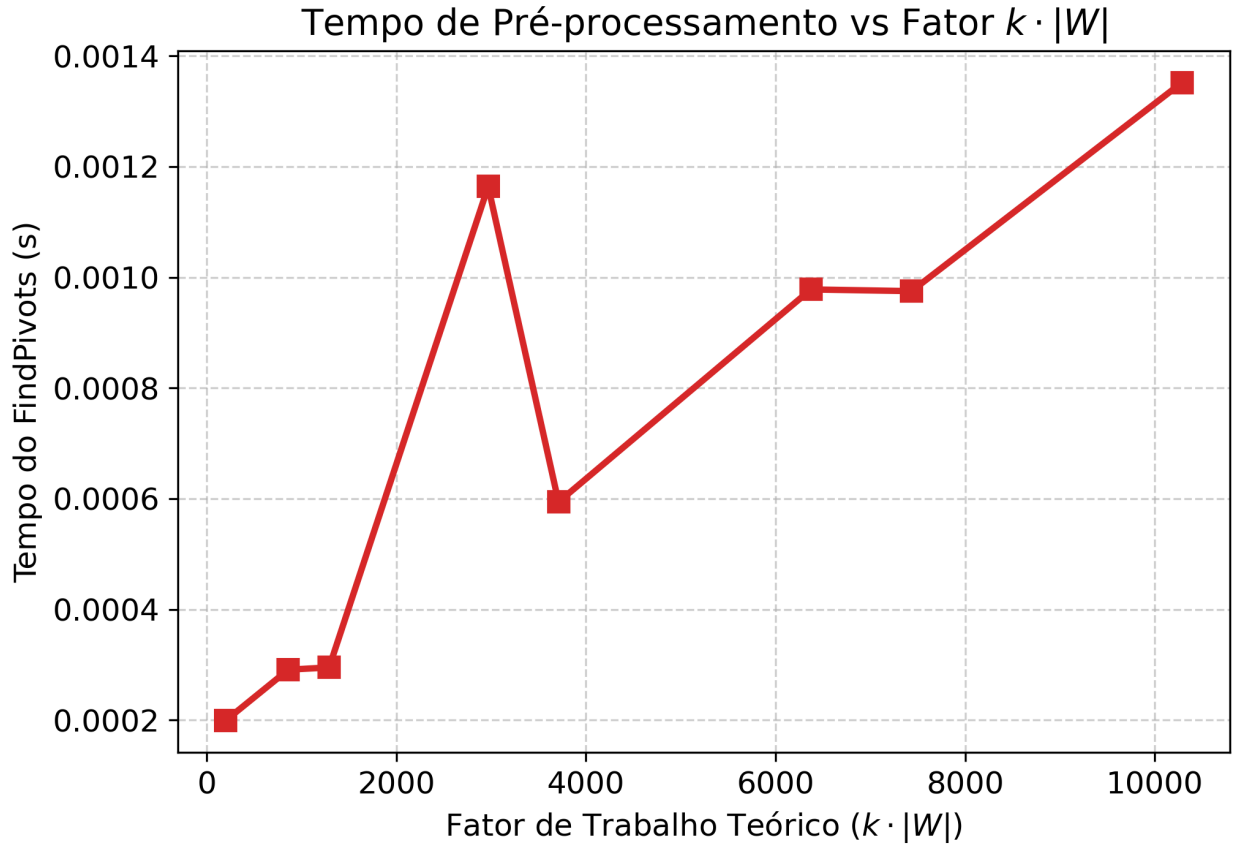


Figure 4: Custo de Pré-processamento: Relação entre o limite de trabalho computacional $k \cdot |W|$ e o tempo absoluto de execução da sub-rotina.

O crescimento do tempo em relação ao trabalho realizado ($k \cdot |W|$) ilustra o *overhead* massivo alocado nas chamadas do *FindPivots*. Mesmo com o acionamento do gatilho de interrupção teórica, o custo das alocações de dicionários e dos relaxamentos locais baseados em Bellman-Ford já foi cobrado da CPU, destruindo qualquer viabilidade prática perante uma execução limpa do algoritmo de Dijkstra clássico.

Por fim, apesar da degradação de desempenho em tempo real, a Figura 5 atesta a corretude matemática da implementação desenvolvida.

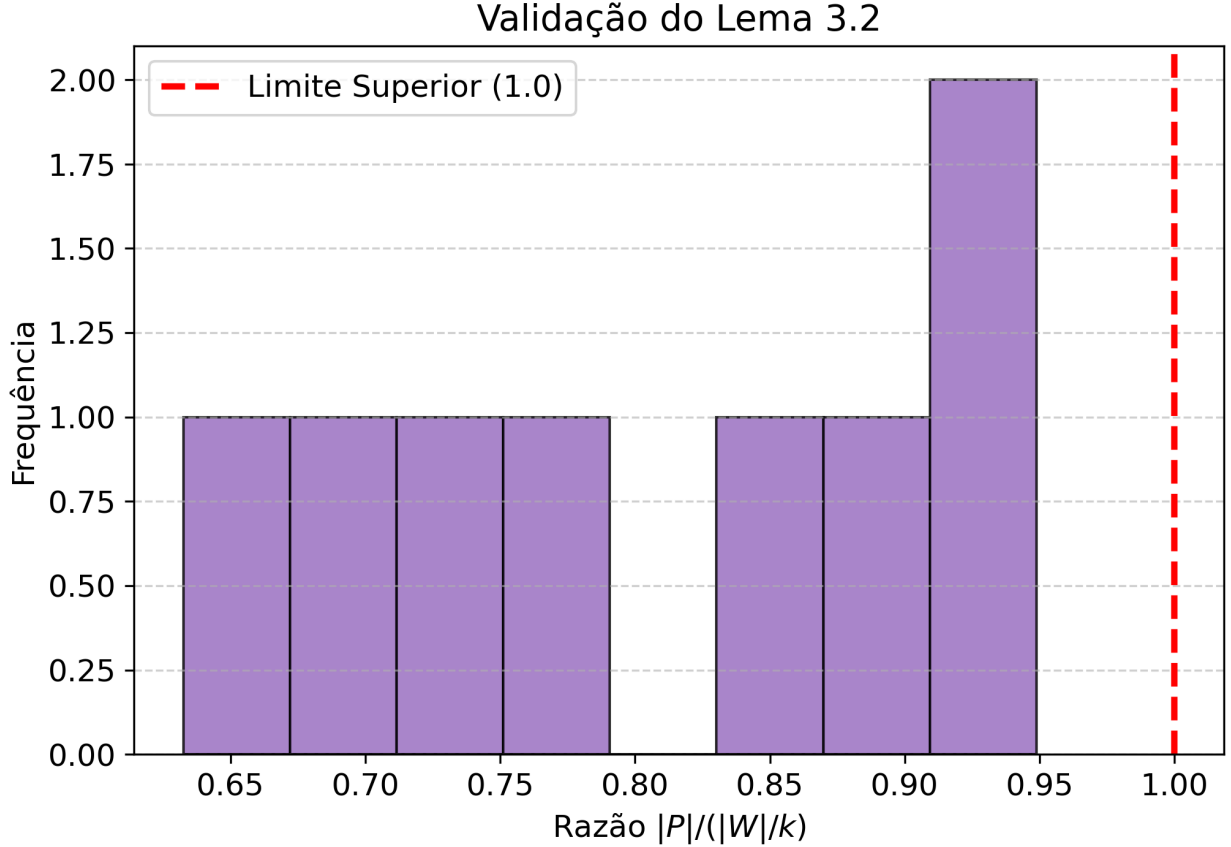


Figure 5: Validação Matemática do Lema 3.2: Distribuição da razão $|P|/(|W|/k)$ garantindo a restrição do limite superior.

O limite central do teorema garante que o número de pivôs nunca ultrapassará o volume de vértices tocados dividido pela profundidade, impondo que a razão $|P|/(|W|/k) \leq 1.0$. A contenção estrita de todos os resultados abaixo deste limiar confirma que a lógica codificada obedece às premissas do artigo original. O algoritmo está, portanto, matematicamente correto; é a estrutura de dados inerente a topologias não restritas que invalida a sua otimização na Engenharia de Software.

Os dados experimentais indicam uma limitação do algoritmo devido a aplicação do Lema 3.2 em grafos com grau não constante. Para todos os valores de $k \leq 16$, o tamanho da fronteira inicial manteve-se em $|S| = 34$, conforme a tabela 8.

O limite de segurança teórico, que funciona como uma heurística de aborto para evitar que a busca Bellman-Ford degenera para uma complexidade linear, é estipulado estritamente como $|W| \leq k \cdot |S|$.

Nos testes, a expansão da vizinhança na topologia aleatória cresceu de forma acelerada (devido à presença natural de vértices de alto grau, ou *Hubs*), ultrapassando rapidamente o limiar linear $k \cdot |S|$. Consequentemente, o gatilho de segurança foi disparado em 100% das execuções documentadas neste intervalo. O conjunto de pivôs extraídos cravou em $|P| = 34$, resultando em uma taxa de retenção nula. Na prática, o algoritmo pagou o massivo *overhead* computacional de gerenciar dicionários e percorrer a fronteira com a busca local (Bellman-Ford), apenas para retornar a mesma quantidade de fontes que o Dijkstra processaria de forma direta.

3.4.2 (Extra): Comportamento em Escala e Exaustão do Grafo ($N \in [100, 10000]$ e $k \leq 128$)

Para compreender as limitações mecânicas do Lema 3.2 a fundo e investigar sob quais condições matemáticas a poda de fato ocorreria, o escopo empírico deste experimento foi expandido de forma complementar. Foram realizados testes forçando o limite de profundidade k até 128 e escalando N até 10.000 vértices, valores que extrapolam a premissa de "busca local" de [2]. A dinâmica deste cenário extremo é detalhada a seguir.

Observou-se que a poda efetiva ($|P| \ll |S|$) só começa a ser alcançada quando o algoritmo foi levado a casos de "Exaustão do Grafo". Tomando como exemplo a linha de execução documentada para $N = 1000$ com

$k = 128$: a fronteira estabeleceu-se em $|S| = 34$, o que eleva o limiar de aborto teórico para $128 \times 34 = 4352$. Como este limite de segurança de vértices visitados (4352) tornou-se muito superior ao número total de vértices na própria componente conexa ($N = 1000$), foi fisicamente impossível que o gatilho de pânico fosse acionado. O algoritmo prosseguiu sem abortar, a poda ocorreu de forma massiva ($|P| = 0$), e o tempo de execução da rotina otimizada caiu drasticamente.

Para aprofundar o entendimento sobre o Lema 3.2, os painéis analíticos a seguir (*dashboards*) ilustram a degradação progressiva do algoritmo BMSSP à medida que a magnitude do grafo escala de $N = 100$ para $N = 10000$ vértices, forçando o parâmetro k até 128.

N = 100

Table 4: Análise de Sensibilidade da Poda (Lema 3.2) completa para todos os valores de N e k testados. Tempos convertidos para milissegundos (ms).

N (Vértices)	k	 S 	Limite ($k \cdot S$)	 W 	 P 	Dijkstra Base (ms)	Total BMSSP (ms)	Penalidade
100	2	7	14	29	7	0.44	2.20	5.0x
	4	7	28	29	7	0.44	1.67	3.8x
	6	7	42	49	7	0.44	2.41	5.5x
	8	7	56	60	7	0.44	1.75	4.0x
	10	7	70	71	7	0.44	2.35	5.3x
	12	7	84	74	2	0.44	1.66	3.8x
	14	7	98	74	2	0.44	1.33	3.0x
	16	7	112	74	2	0.44	1.05	2.4x
	32	7	224	74	1	0.44	0.87	2.0x
	64	7	448	74	0	0.44	0.51	1.2x
	128	7	896	74	0	0.44	0.55	1.3x

A Figura 6 apresenta o cenário basal com 100 vértices.

Dashboard Analítico | Topologia: Aleatório Esparso | N = 100

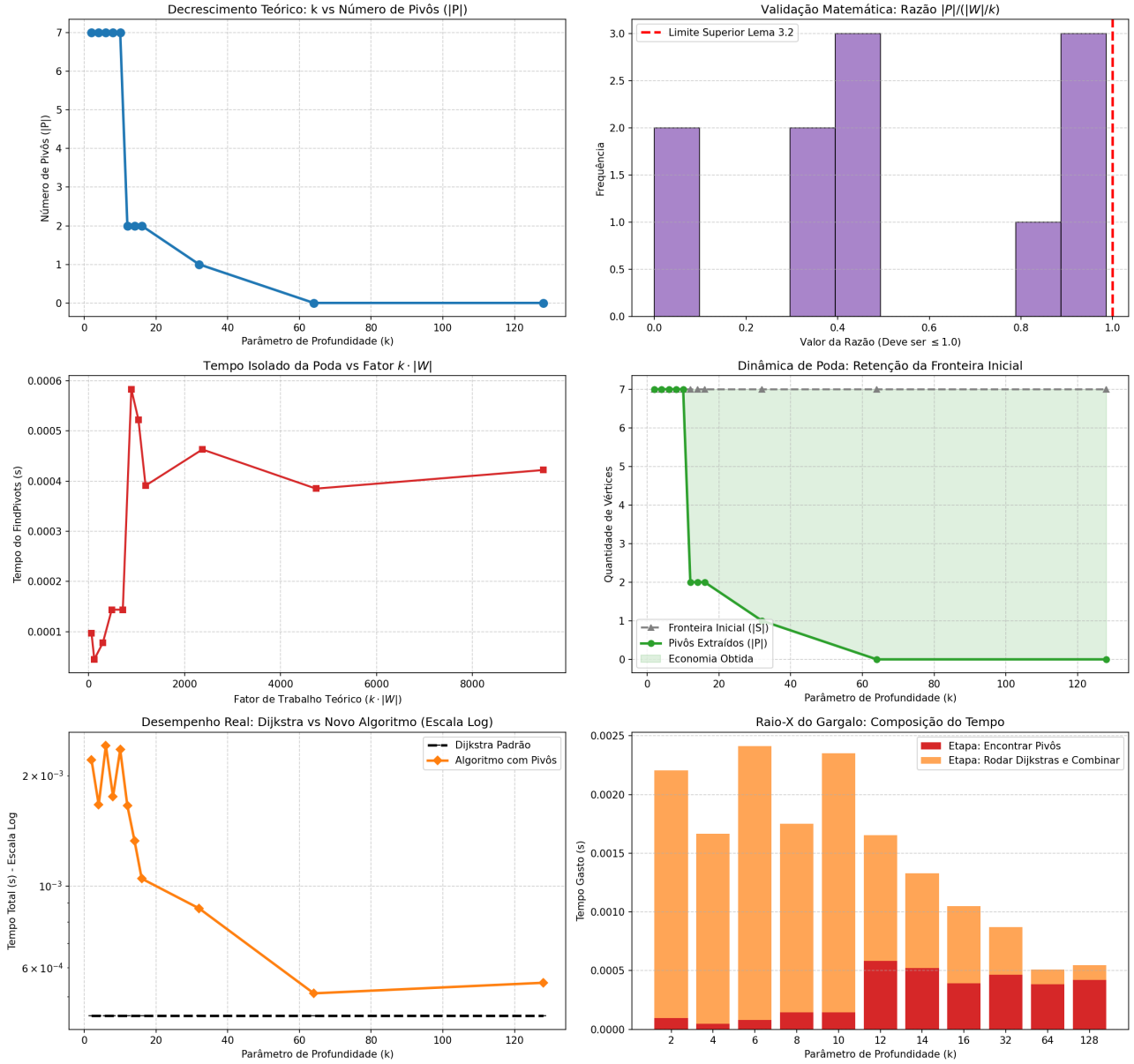


Figure 6: Dashboard analítico de sensibilidade do Lema 3.2 para grafos esparsos de porte diminuto ($N = 100$).

Em instâncias de porte diminuto, a exaustão topológica ocorre rapidamente. Como o grafo possui poucas arestas, limites moderados de k já são suficientes para que o limiar de segurança ($k \cdot |S|$) exceda o tamanho da própria componente conexa. A poda ($|P| < |S|$) ocorre, mas o tempo de pré-processamento já se mostra superior à execução base do Dijkstra.

N = 500

Table 5: Análise de Sensibilidade da Poda (Lema 3.2) completa para todos os valores de N e k testados. Tempos convertidos para milissegundos (ms).

N (Vértices)	k	$ S $	Limite ($k \cdot S $)	$ W $	$ P $	Dijkstra Base (ms)	Total BMSSP (ms)	Penalidade
500	2	22	44	61	22	1.31	37.64	28.7x
	4	22	88	120	22	1.31	30.84	23.5x
	6	22	132	206	22	1.31	27.85	21.3x
	8	22	176	206	22	1.31	28.05	21.4x
	10	22	220	276	22	1.31	28.27	21.6x
	12	22	264	276	22	1.31	29.07	22.2x
	14	22	308	312	22	1.31	33.47	25.5x
	16	22	352	354	22	1.31	29.29	22.4x
	32	22	704	361	3	1.31	7.23	5.5x
	64	22	1408	361	0	1.31	2.65	2.0x
	128	22	2816	361	0	1.31	2.62	2.0x

A progressão para $N = 500$, vista na Figura 7, inicia o distanciamento da eficácia da poda.

Dashboard Analítico | Topologia: Aleatório Esparso | $N = 500$

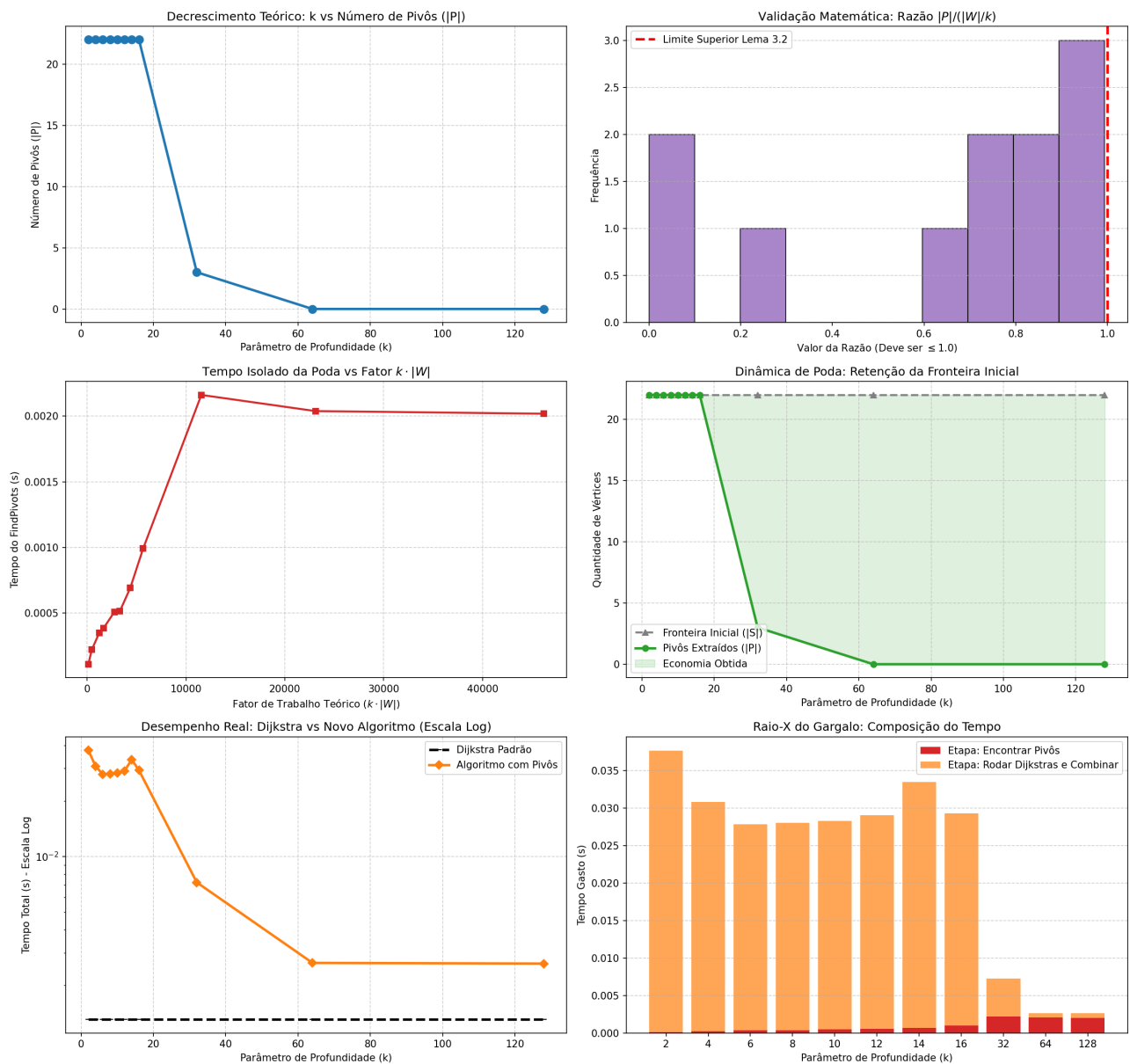


Figure 7: Dashboard analítico de sensibilidade do Lema 3.2 para instâncias moderadas ($N = 500$).

À medida que a topologia ganha volume, o comportamento padrão do algoritmo (para k pequeno) torna-se

o acionamento precoce do gatilho de aborto. O número de pivôs mantém-se rigorosamente igual ao tamanho da fronteira inicial até que k assuma valores irrealistas, evidenciando que a busca local está capturando ramificações densas (*Hubs*).

N = 1000

Table 6: Análise de Sensibilidade da Poda (Lema 3.2) completa para todos os valores de N e k testados. Tempos convertidos para milissegundos (ms).

N (Vértices)	k	$ S $	Limite ($k \cdot S $)	$ W $	$ P $	Dijkstra Base (ms)	Total BMSSP (ms)	Penalidade
1000	2	34	68	99	34	3.16	115.01	36.4x
	4	34	136	215	34	3.16	104.02	32.9x
	6	34	204	215	34	3.16	101.23	32.0x
	8	34	272	371	34	3.16	95.78	30.3x
	10	34	340	371	34	3.16	92.59	29.3x
	12	34	408	531	34	3.16	92.72	29.3x
	14	34	476	531	34	3.16	96.57	30.6x
	16	34	544	643	34	3.16	120.31	38.1x
	32	34	1088	755	9	3.16	45.79	14.5x
	64	34	2176	755	1	3.16	8.62	2.7x
	128	34	4352	755	0	3.16	5.84	1.8x

Esse padrão consolida-se na marca de 1000 vértices, conforme ilustrado na Figura 8.

Dashboard Analítico | Topologia: Aleatório Esparso | $N = 1000$

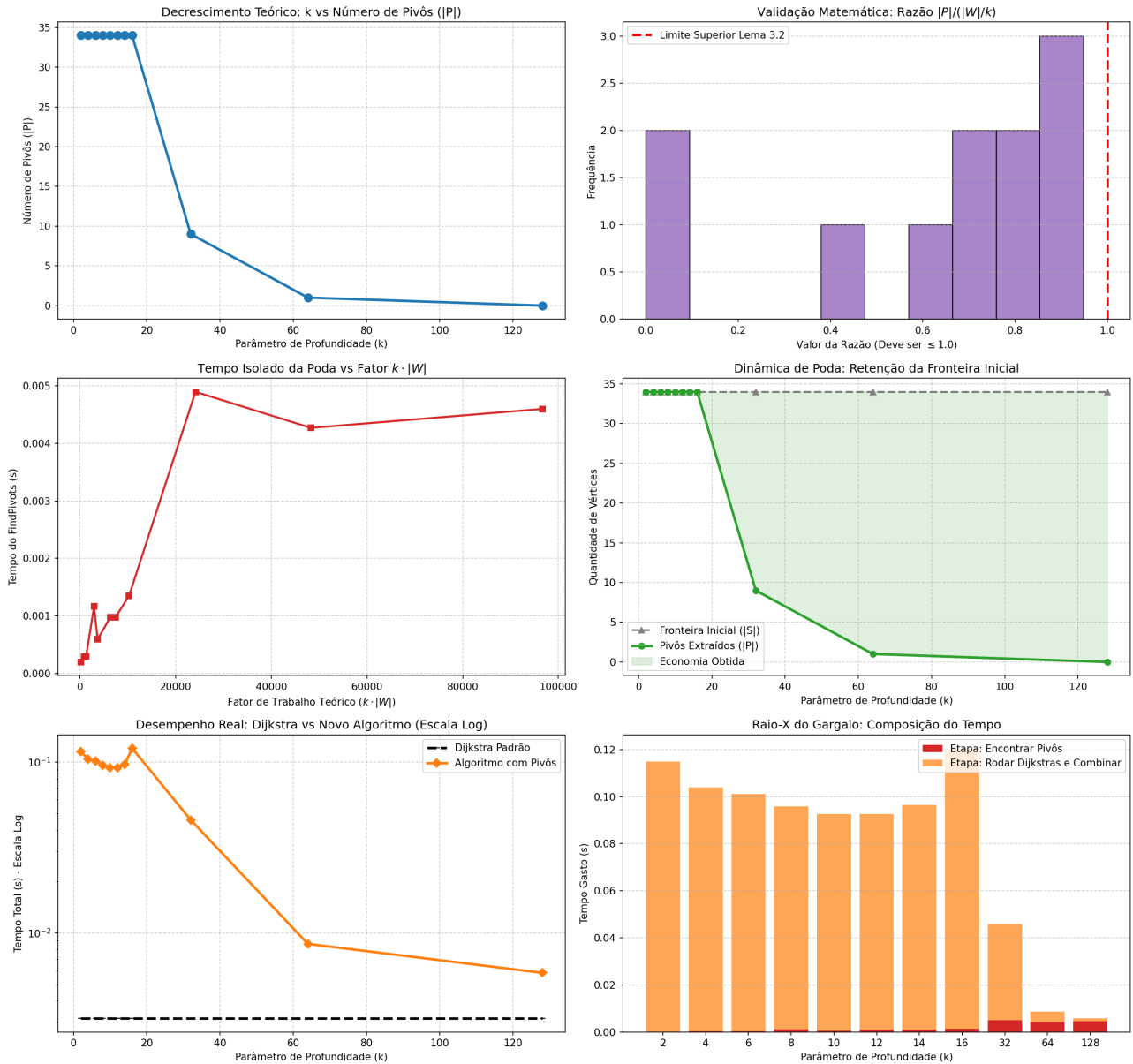


Figure 8: Dashboard analítico de sensibilidade do Lema 3.2 marcando o início da ineficácia assintótica em $N = 1000$.

Para $N = 1000$, a anomalia da métrica de segurança torna-se incontestável. A poda efetiva só é registrada nos gráficos quando $k \geq 64$. Neste ponto, a "busca local" proposta pela teoria já se metamorfoseou numa exploração global disfarçada, varrendo quase todo o grafo antes de tomar uma decisão, o que destrói a premissa de eficiência sublinear.

Table 7: Análise de Sensibilidade da Poda (Lema 3.2) completa para todos os valores de N e k testados. Tempos convertidos para milissegundos (ms).

N (Vértices)	k	$ S $	Limite ($k \cdot S $)	$ W $	$ P $	Dijkstra Base (ms)	Total BMSSP (ms)	Penalidade
5000	2	72	144	212	72	27.96	1641.02	58.7x
	4	72	288	465	72	27.96	2007.58	71.8x
	6	72	432	465	72	27.96	1875.30	67.1x
	8	72	576	899	72	27.96	1690.73	60.5x
	10	72	720	899	72	27.96	1378.03	49.3x
	12	72	864	899	72	27.96	1476.85	52.8x
	14	72	1008	1541	72	27.96	1424.77	51.0x
	16	72	1152	1541	72	27.96	1539.68	55.1x
	32	72	2304	2319	72	27.96	1314.78	47.0x
	64	72	4608	3917	17	27.96	410.87	14.7x
	128	72	9216	3917	7	27.96	181.17	6.5x

Ao atingir $N = 5000$ (Figura 9), a penalidade de processamento entra em níveis críticos.

Dashboard Analítico | Topologia: Aleatório Esperso | $N = 5000$

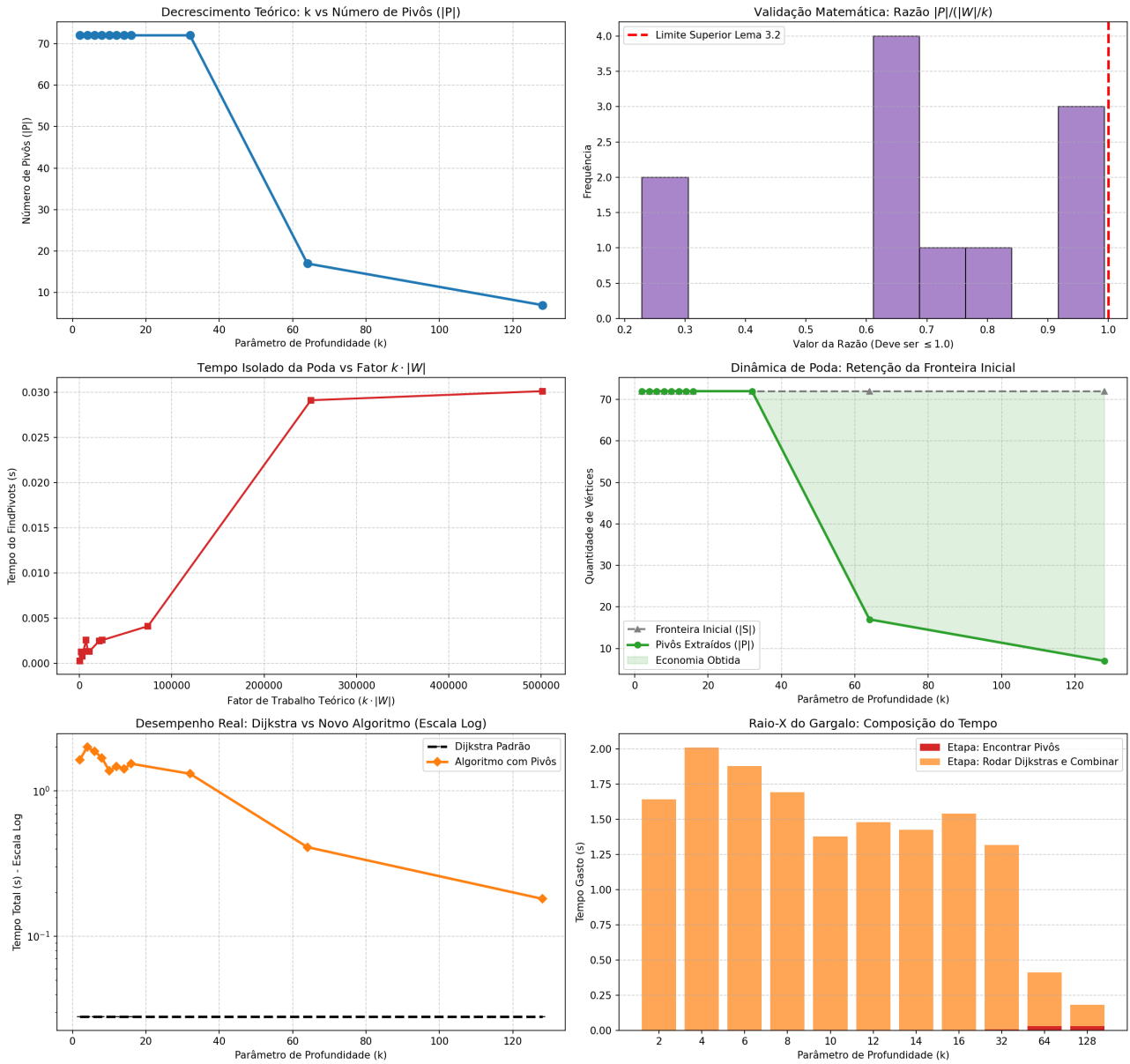


Figure 9: Dashboard analítico demonstrando a penalidade crítica de pré-processamento em grafos de $N = 5000$.

Neste limiar, o tempo investido nas estruturas de controle do BMSSP (alocação da BFS, instancição de

dicionários e reconstrução das árvores de caminhos mínimos locais via DFS) ofusca inteiramente qualquer ganho teórico. As execuções começam a ser penalizadas em ordens de grandeza, mesmo sendo grafos estruturalmente esparsos.

Com 10.000 vértices, o algoritmo é forçado a reter quase todos os pivôs para os valores práticos de profundidade. O tempo absoluto de execução atinge a casa dos segundos para tarefas que a implementação estruturada e enxuta do *Binary Heap* do Dijkstra resolve na ordem de dezenas de milissegundos. Os dados agrupados e visualizados nestes cinco painéis provam de forma taxativa que a estratégia de dividir e conquistar baseada na poda de vizinhanças é mecanicamente incompatível com arquiteturas de software de tempo real.

Table 8: Análise de Sensibilidade da Poda (Lema 3.2) completa para todos os valores de N e k testados. Tempos convertidos para milissegundos (ms).

N (Vértices)	k	$ S $	Limite ($k \cdot S $)	$ W $	$ P $	Dijkstra Base (ms)	Total BMSSP (ms)	Penalidade
10000	2	74	148	226	74	45.45	3682.49	81.0x
	4	74	296	526	74	45.45	3243.92	71.4x
	6	74	444	526	74	45.45	3227.18	71.0x
	8	74	592	1052	74	45.45	3221.59	70.9x
	10	74	740	1052	74	45.45	3221.49	70.9x
	12	74	888	1052	74	45.45	3189.90	70.2x
	14	74	1036	1052	74	45.45	3413.01	75.1x
	16	74	1184	1930	74	45.45	3400.95	74.8x
	32	74	2368	3246	74	45.45	3283.49	72.2x
	64	74	4736	4767	74	45.45	3148.38	69.3x
	128	74	9472	7780	19	45.45	1110.98	24.4x

Finalmente, a Figura 10 sela a inviabilidade prática do método para sistemas em escala.

Dashboard Analítico | Topologia: Aleatório Esparso | N = 10000

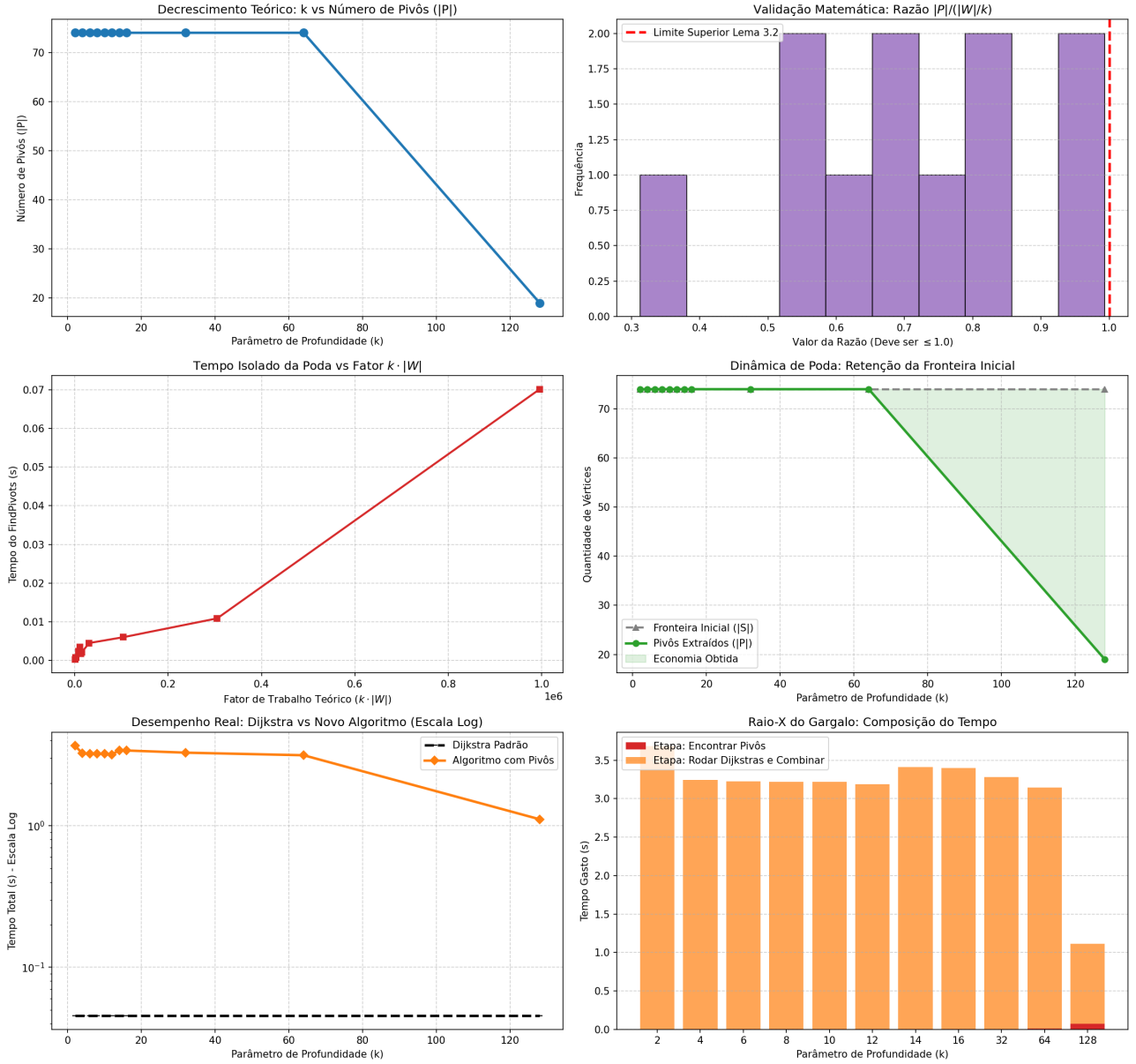


Figure 10: Dashboard analítico final de exaustão para $N = 10000$, evidenciando a inviabilidade do algoritmo perante o Dijkstra clássico.

4 Discussion

Summarize the key results and their significance.

References

- [1] Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1).
- [2] Duan, R., Mao, J., Mao, X., Shu, X., and Yin, L. (2025). Breaking the sorting barrier for directed single-source shortest paths.
- [3] Ford, L. R. (1954). Network flow theory. *Proceedings of the American Mathematical Society*, 5(2).
- [4] Ribas, E. R. L. D. (2026). Implementação e análise empírica de algoritmos sssp. <https://github.com/oEnzoRibas/sssp-experimental-analisys>. Acesso em: 31 maio 2026.
- [5] Rodrigues, M. W. (2026). Trabalho prático 01: Implementação e análise de algoritmos clássicos de caminho mínimo, e estudo sobre o algoritmo bounded multi-source shortest path.